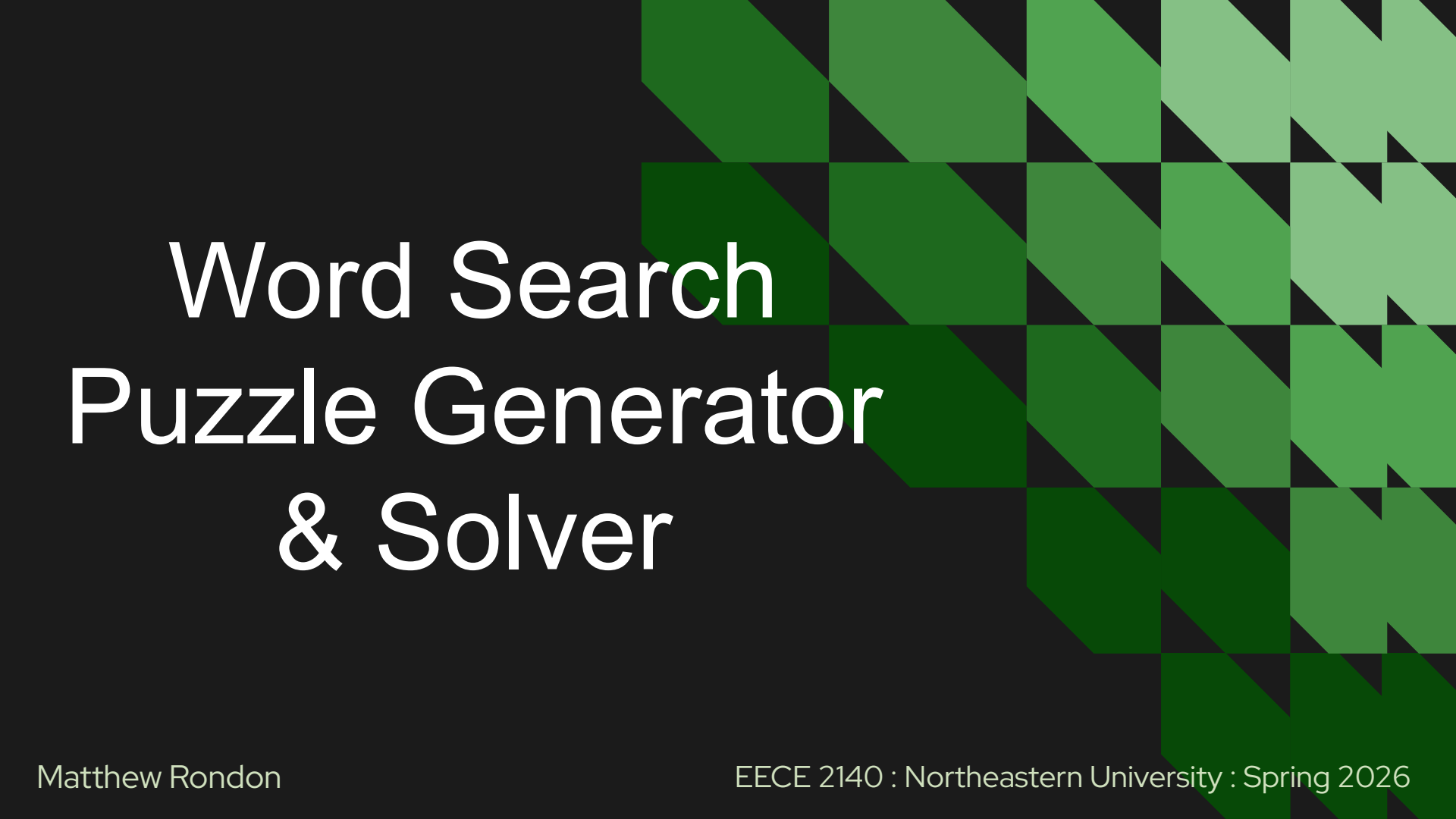




Word Search Puzzle Generator & Solver

Project overview:

Why I chose this project



- Word search puzzles are **widely known**
- Uses **Functions** and **program structure**
- Utilizes a **2D grid**
- Uses **Vectors** to store letters and words
- Includes **randomization**
- Applies in class techniques like:
 - **Backtracking**
 - **Search Algorithms**
 - **Object Oriented Programming**

Project Objectives

Functionality

- Generate a word search puzzle from a list of words
- Place words in a variety of different directions (Horizontal, Vertical, Diagonal)
- Develop a program that can locate words in the grid

Implementation

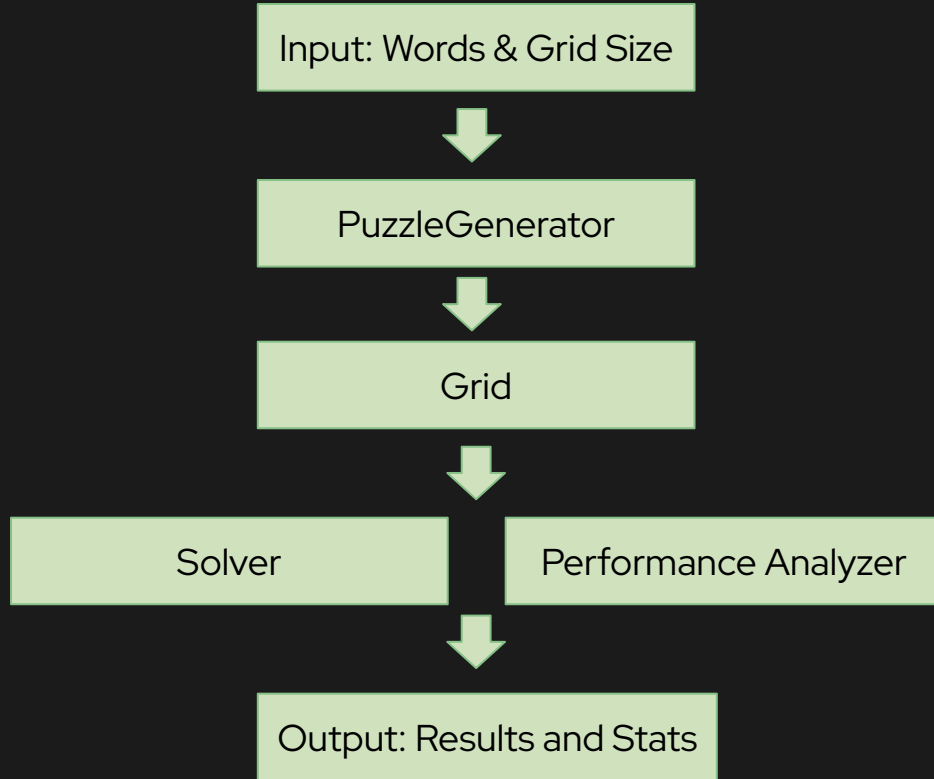
- Use Object-Oriented Programming (OOP)
- Use vectors, and references to manage data gathered
- Apply backtracking for word placement
- Ensure valid placement and allow letter overlapping

Measurables

- Track and record data to analyze user production
 - Time it takes
 - Number of placements attempts



System Architecture



User Input: Word list and grid sizing

Grid Class: Stores and manages the 2D puzzle

PuzzleGenerator (Class): Places words using backtracking

PuzzleSolver (Class): Searches for and finds words

Performance analyzer: measures key data like runtime and efficiency

Output: completed puzzle and solution results

Puzzle Generator

Initializes an empty 2D grid

Takes the input list of words

Places words into grid:

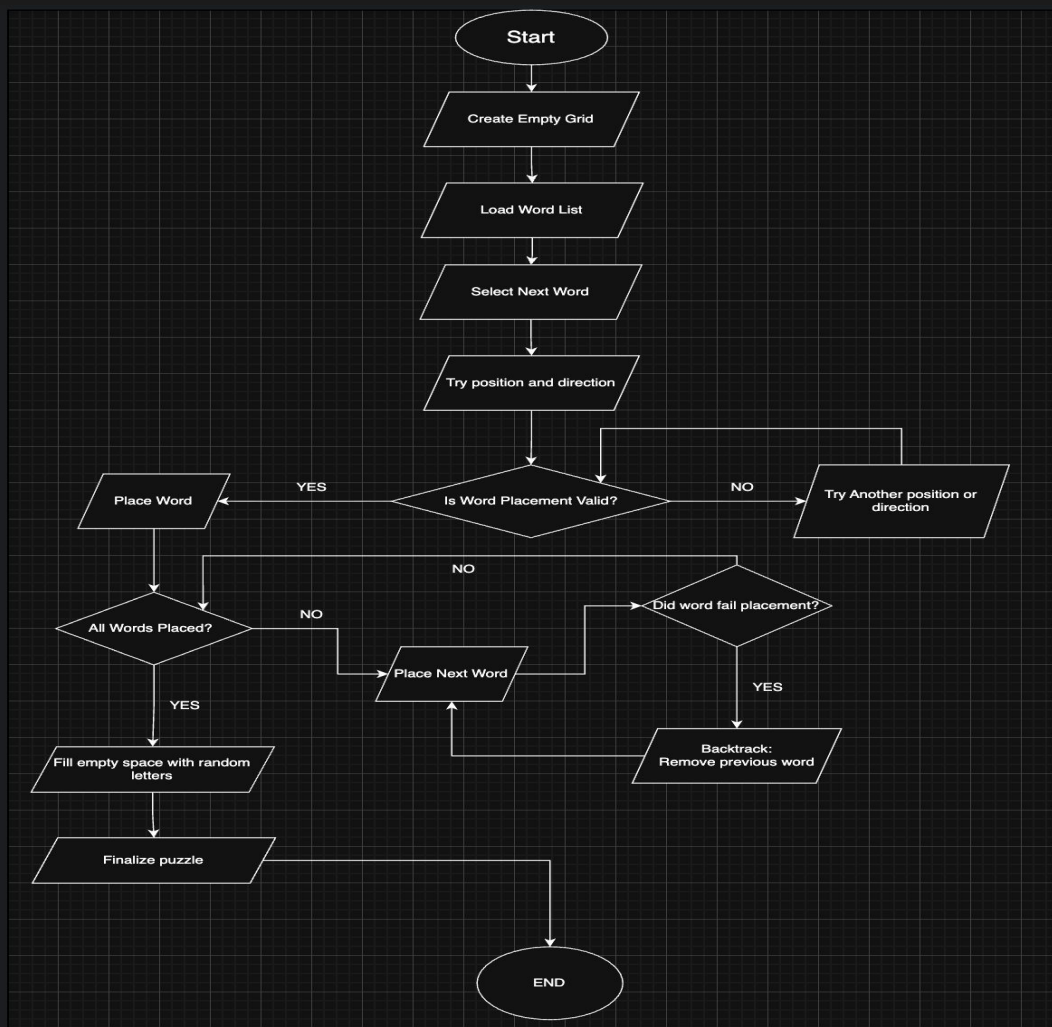
- Horizontal
- Vertical
- Diagonal

Checks for **Valid Placement**:

- Stays in bounds
- No conflicting letters
- Allows matching overlaps

Utilizes backtracking if placement fails

Fills remaining space with random letters



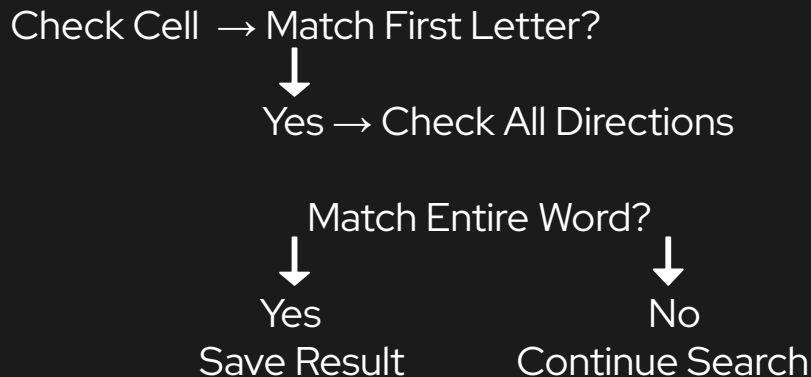
Backtracking Algorithm

- Used when unsuccessful placement of a word
- Tries all possible positions and directions
- If placement fails it undoes previous word
- Retries in a different placement
- Will run until all words are placed or no valid solutions exist
- Ensures a valid puzzle configuration

```
function placeWords(index):  
    if index == totalWords:  
        return true  
  
    word = words [index]  
  
    for each position in grid:  
        for each direction:  
            if placement is valid:  
                place word  
                if placeWords(index + 1):  
                    return true  
  
                remove word // backtrack  
  
    return false
```

Puzzle Solver

- Searches for each word in the 2D grid
- Scans every row and column
- Matches the first letter of the word
- Searches in **8 possible directions**:
 - Horizontal
 - Vertical
 - Diagonal
- Compares letters one by one
- Stops if mismatch occurs
- Records:
 - Starting position
 - Direction of the word



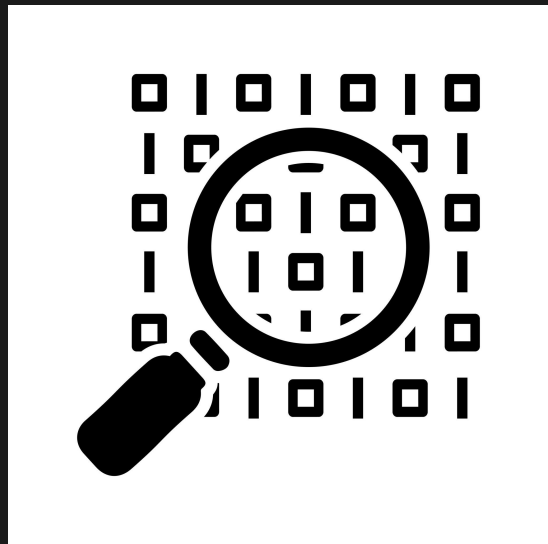
Data Structures and C++ Concepts

Data Structures

- `vector<vector<char>>` used to store the 2D grid
- `vector<string>` used to store the word list

C++ Concepts Applied

- Classes (Grid, Generator, Solver)
- Functions for modular design
- References to pass large data efficiently
- Loops and conditionals for grid scanning



Challenges Faced

- Placing words without conflicts
- Handling the placement and scanning of multiple directions
- The implimentation of backtracking
- Grid boundaries (out of bounds errors)



Performance Results

- Measured runtime and solving time for each run
- Tested different grid sizes:
 - 10x10
 - 15x15
 - 20x20
- The larger the grid the slower the runtime and solving time
- More words also meant increased placement difficulties and more backtracking

Demo / Output shell

Conclusion

- Successfully built a word-search puzzle generator and solver in C++
- Applied key concepts learned in class throughout the semester
 - vectors
 - OOP
 - Backtracking
- Learned problem solving skills breaking problems into smaller workable solutions

